

Compiler Design

Theoretical Question Answer

1. What is a compiler? Explain the main purpose of a compiler in a programming language environment.

Ans: A **compiler** is a computer program (or set of programs) that

- Transforms source code written in a programming language
- Is a translator program



The main purpose of a compiler in a programming language environment.

- To design a translator
- Influences on programming language design
- Influences on computer design
- Compiling techniques are useful for software development
 - Parsing techniques are often used
 - Learn practical data structures and algorithms
 - Basis for many tools such as text formatters, structure editors, design verification tools,...

Source Code → Compiler → Object Code / Machine Code → Execution

2. Differentiate between a compiler and an interpreter.

Compiler

Interpreter

Translates the **whole program at once**.

Translates **one line at a time**.

Generates an **executable file**.

Does **not create** an executable file.

Finds errors **after compiling the whole program**.

Finds errors **line by line immediately**.

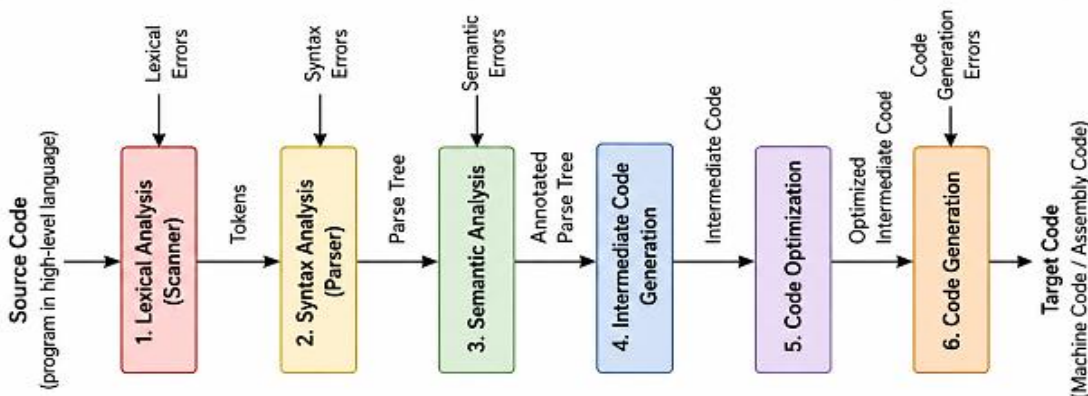
Program execution is **faster** after compilation.

Program execution is **slower** because translation happens during running.

Example: C Programming Language, C++

Example: Python, JavaScript

3. What are the various phases of a compiler? Briefly explain the function of each phase with a proper figure.



Explain the each function:

i. **Lexical analysis:** Experience with **Regular Expressions** and **will be important** when you deal with

- big text files;
- automating routine text processing tasks.

ii. **Syntax Analysis:** Parsing is important in

- Text and language processing tasks; if you're trying to interpret tweets or sentences, etc.

iii. **Semantic Analysis:**

- At this stage, actually start to understand the programming languages that use even in our day to day life.
- Understanding various constraints related to assignment, allocation

iv. Intermediate Code Generation:

- Generates middle level code according to the semantics.
- You will learn different platform independent codes

v. Code Optimization:

- introduced to various **Optimization** techniques in code generation.
- also be more critical of the code write for other works.

vi. Code Generation:

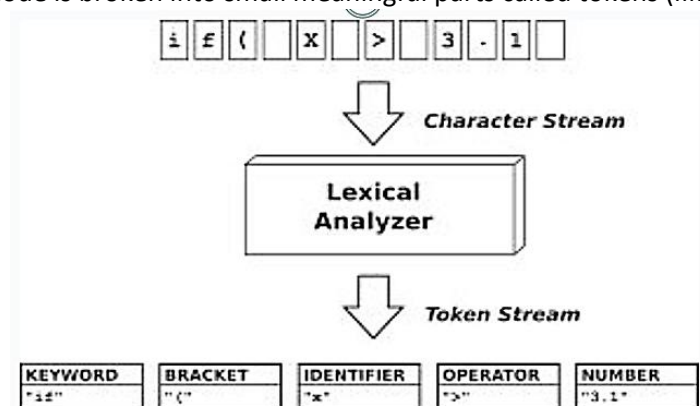
- learn to generate the **Machine Code**.
- This forces to learn Assembly Language;
- an overview of applying basic Computer Architecture concepts.

4. What is lexical analysis? What is the role of a lexical analyzer in a compiler?

Lexical analysis is the first step in a compiler where the source code is broken into small meaningful parts called tokens (like words, numbers, and symbols).

The role of Lexical analyzer:

- ❖ First phase of a compiler
- ❖ Main task
 - ✓ To read the input characters
 - ✓ To produce a sequence of **tokens**
 - ✓ An assistant of parser



5. Define the following terms with examples:

a. Token

b. Lexeme

Ans: a) Token: A **token** is a small meaningful unit of a program in a programming language. Types of words in source program like Keywords, operators, identifiers, constants, literal strings, punctuation symbols (such as commas, semicolons).

Example:

- Keywords: int, if, return
- Identifiers: sum, x, total
- Operators: +, -, =
- Symbols: ;, {}, {}
- Numbers: 10, 25

b) Lexeme: Actual words in source program matched by pattern. It's a valid token.

Example: int, sum, =, 10

6. What is syntax analysis (parsing)? Why is it important in the compilation process?

Syntax Analyzer creates the syntactic structure of the given source program. This syntactic structure is mostly a parse tree. The syntax of a programming is described by a *context-free grammar (CFG)* or BNF (Backus-Naur Form) notation.

Why it is important:

- I. Checks program structure
- II. Finds syntax errors
- III. Builds parse tree
- IV. Helps later phases

7. What is context-free grammar (CFG)? Explain the different components of a CFG with an example.

A **Context-free Grammar (CFG)** is utilized to describe the Syntactic Structure of a Language.

Components of CFG: A CFG is written as a 4 tuple like that $G = T, N, P, S$

1. Terminals(T): These are the actual symbols of the language. They cannot be broken further.

Example: id, +, -, *, /, (,)

2. Non-terminals / Variables (N/V): These are symbols that can be replaced further. They help define structure.

Example: *expr*, *op*

3. Productions(P): These are the rules used to replace variables.

Example: $expr \rightarrow expr\ op\ expr$, $expr \rightarrow (expr)$, $expr \rightarrow -\ expr$, $expr \rightarrow id$, $op \rightarrow + \mid - \mid * \mid /$

4. Start Symbol(S): The starting point of the grammar. Usually represented by *s*.

Example: *expr*

8. What is the purpose of a parse tree?

- Checks syntax correctness
- Shows program structure
- Helps detect syntax errors
- Helps semantic analysis
- Supports code generation

$list \rightarrow list + digit$

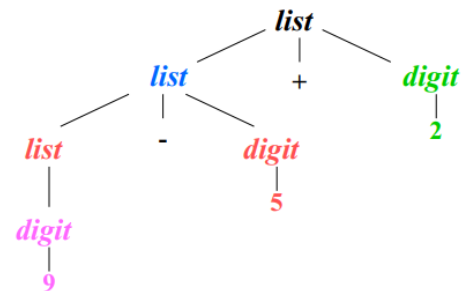
$\rightarrow list - digit + digit$

$\rightarrow digit - digit + digit$

$\rightarrow 9 - digit + digit$

$\rightarrow 9 - 5 + digit$

$\rightarrow 9 - 5 + 2$



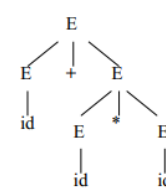
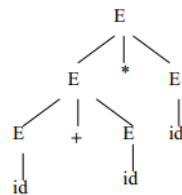
9. Define Ambiguity in CFG with proper examples.

A grammar produces more than one parse tree for a sentence is called as an **ambiguous** grammar. A **CFG is ambiguous** if one string can have more than one parse tree.

Example:

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E$
 $E \rightarrow (E)$
 $E \rightarrow id$

$E \Rightarrow E * E$
 $\Rightarrow E + E * E$
 $\Rightarrow id + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$



$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

Technical Questions

1. Given a piece of code and the CFG, divide the program into appropriate lexeme. Slide-2, Page: 27-32

i.

```
float limitedSquare(x)
{ float x
  return (x <= -10.0 || x >= 10.0) ? 100 : x*x;
}
```

Answer

`<float> <id, limitedSquare> <(> <id, x> <(> <{> <float> <id, x> <return> <(> <id, x> <op, "<="> <num, -10.0> <op, "||> <id, x> <op, ">="> <num, 10.0> <(> <op, "?"> <num, 100> <op, ":"> <id, x> <op, "*"> <id, x> <(>`

ii. Tagged languages like HTML or XML are different from conventional programming languages in that the tags are either very numerous (as in HTML) or a user-definable set (as in XML). Further, tags can often have parameters. Suggest and divide the following HTML document into appropriate lexemes.

`Here is a photo of <b>my house</b>; <p><br/> see <a href="morePix.html">More Picture</a> if you liked that one.</p>`

Answer

`<text, "Here is a photo of"> <nodestart, b> <text, "my house"> <nodeend, b> <nodestart, p> <selfendnode, img> <selfendnode, br> <text, "see"> <nodestart, a> <text, "More Picture"> <nodeend, a> <text, "if you liked that one."> <nodeend, p>`

2. Write the regular expressions for matching given types of strings. Slide-2, Page: 14-26

String Matching: i. Username RegEx, ii. Password RegEx, iii. Email Address RegEx, iv. Validate Date

i. Username RegEx:

- Size ranging from 3 to 12
- Can contain small alphabets and digits
- Expression
 - $[a-z0-9]\{3, 12\}$

ii. Password RegEX:

- Size greater than 8
- Contain alphabet and digits
- Expression
 - $[a-zA-Z0-9]\{8,\}$
- Can contain special character
 - $[a-zA-Z0-9@#\%]\{8,\}$

iii. Email Address RegEx:

- Contains @ and .
- Contains host eg gmail.com, pia.aero, github.io
- Contains username eg. P146011
 - Range 4 to 24
- Expression
 - $[a-zA-Z0-9]\{4, 24\}@[a-z0-9\.-]\{2, 4\}$
 - Work for most email.

iv. Valid Date RegEx:

- 31-11-1999
 - Expression: $[0-9]\{1, 2\}-[0-9]\{1, 2\}-[0-9]\{4\}$
 - Validates: 1-1-2000, 07-10-2016 ...
 - Problem...
 - $0[0-9]\{1\}[12]$
 - for year
 - $0[1-9]\{1\}[12][0-9]\{3\}[01]$
 - for month
 - $(19|20)[0-9]\{2\}$ from year ranging
 - 1900-2099

3. Given a CFG, check if ambiguous. Slide-3, Page: 18-24

Ambiguous: $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E$
 $E \rightarrow (E)$
 $E \rightarrow id$

Case 1: $(id + id) * id$

Case 2: $id + (id * id)$

In this grammar:

- There is **no precedence rule** for + and *
- The same expression can be **grouped in different ways**
- Therefore, **multiple parse trees can be formed**

Can be interpreted in two ways:

- First, + is evaluated
- Or, first * is evaluated

This CFG is Ambiguous, because for the same string.

Unambiguous : $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow id$

case : $id + id * id$

In this grammar:

- ✓ * has higher priority than +
- ✓ Fixed grammar rules decide structure
- ✓ Only one valid parse tree exists for each string

$E \Rightarrow E + T$
 $\Rightarrow T + T$
 $\Rightarrow F + T$
 $\Rightarrow id + T$
 $\Rightarrow id + T * F$
 $\Rightarrow id + F * F$
 $\Rightarrow id + id * id$

This CFG is Unambiguous because every valid string has only one unique parse tree and derivation.

End

Grammar:

$stmt \rightarrow |if\ expr\ then\ stmt$
 $|if\ expr\ then\ stmt\ else\ stmt$
 $| \in$
 $expr \rightarrow term\ relop\ term\ | term$
 $term \rightarrow id\ | num$

Assume Following Tokens:

if, then, else, relop, id, num

Regular definitions

$if \leftarrow if$
 $then \leftarrow then$
 $else \leftarrow else$
 $relop \leftarrow < | <= | > | >= | = | <>$
 $id \leftarrow letter\ (letter\ | digit)^*$
 $num \leftarrow digit^+ \cdot (digit^+)^* (E(+ | -)^* digit^+)^*$

Task of recognition of token in a lexical analyzer

Regular expression	Token	Attribute-value
if	if	-
id	id	Pointer to table entry
<	relop	LT

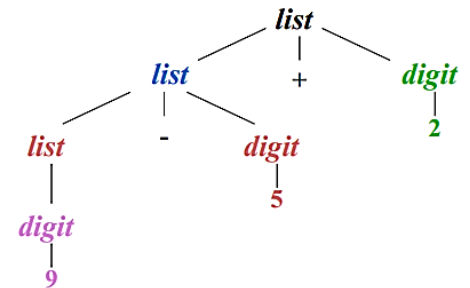
Slide – 03

Page 18-24

Recognition of Tokens

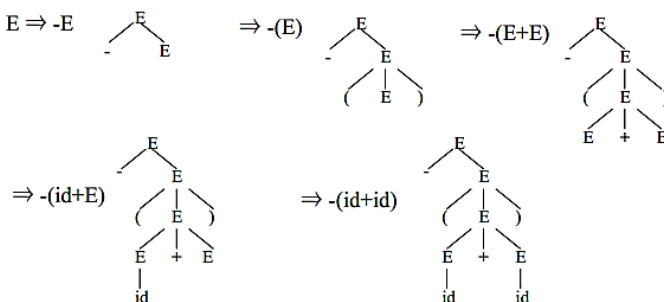
Regular Expression	Token	Attribute-Value
ws	-	-
if	if	-
then	then	-
else	else	-
id	id	pointer to table entry
num	num	pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

Parse Tree

 $list \rightarrow list + digit$ $\rightarrow list - digit + digit$ $\rightarrow digit - digit + digit$ $\rightarrow 9 - digit + digit$ $\rightarrow 9 - 5 + digit$ $\rightarrow 9 - 5 + 2$ 

Parse Tree

- Inner nodes of a parse tree are non-terminal symbols.
- The leaves of a parse tree are terminal symbols.
- Graphical representation of a derivation.



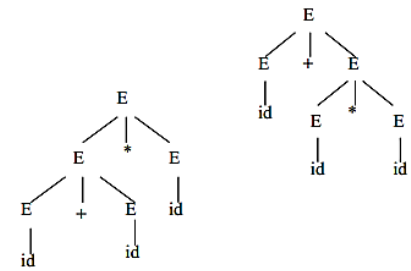
Ambiguity

Ambiguity

A grammar produces more than one parse tree for a sentence is called as an **ambiguous** grammar.

 $E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E$ $E \rightarrow (E)$ $E \rightarrow id$

$E \Rightarrow E * E$
 $\Rightarrow E + E * E$
 $\Rightarrow id + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$



$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

Grammar:

 $string \rightarrow string + string \mid string - string \mid 0 \mid 1 \mid \dots \mid 9$


Final Term

Descriptive

1. Compiler

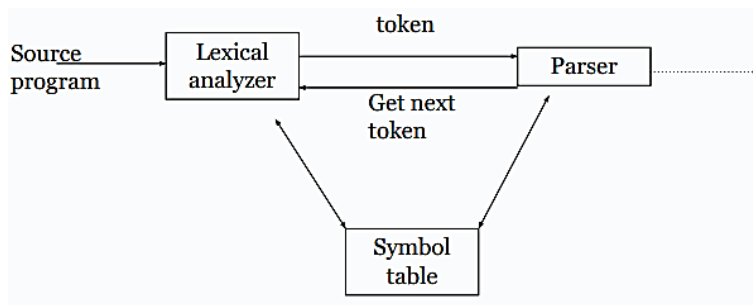
Ans: mid 1, 2, 3

2. Lexical Analyzer

i. Working Process: mid 4

ii. Different Component of Lexical Analyzer.

Ans: A **Lexical Analyzer** is the first phase of a compiler. It reads the source code character by character and converts it into **tokens**.



Example

Token	Informal description	Sample lexemes
if	Characters i, f	if
else	Characters e, l, s, e	else
relation	< or > or <= or >= or == or !=	<=, !=
id	Letter followed by letter and digits	pi, score, D2
number	Any numeric constant	3.14159, 0, 6.02e23
literal	Anything but " surrounded by "	"core dumped"

Main Component:

- ✓ **Scanning:**
 - i. Pre-Processing
 - ii. Correlating error messages from compiler with source program
 - iii. A line number can be associated with an error message
- ✓ **Lexical Analysis**
- ✓ **Token**
 - Keywords, operators, identifiers, constants, literal strings, punctuation symbols (such as commas, semicolons)
- ✓ **Lexeme**
 - i. Actual words in source program matched by pattern
 - ii. valid token
- ✓ **Pattern**
 - i. A rule describing the set of lexemes that can represent a particular token in source program
 - ii. Relation {<, <=, >, >=, ==, <>}

iii. Define the Token.

Ans: mid question 5

3. Syntax Analysis

i. Define Syntax Analysis. Ans: mid 6.

ii. Grammars: CFG

iii. Trees:

a) Define Parse Trees with diagram.

Ans: A parse tree pictorially shows how the start symbol of a grammar derives a string in the language.

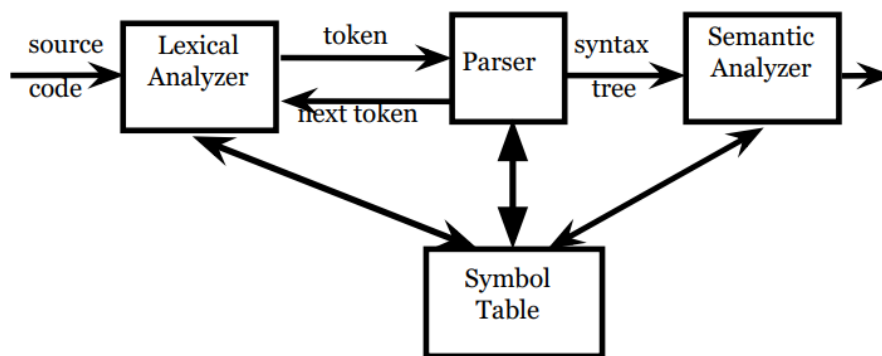


Figure: Parse Tree

b) More Formally, a Parse Tree for a CFG Has the Following Properties:

- ☐ Root Is Labeled with the **Start Symbol**
- ☐ Leaf Node Is a Token or ϵ
- ☐ Interior Node Is a **Non-Terminal**
- ☐ If $A \rightarrow x_1x_2...x_n$, Then A Is an Interior; $x_1x_2...x_n$ are Children of A and May be **Non-Terminals** or **Tokens**

c) Write the purpose of Parse tree.

Ans: mid 8.

iv. Ambiguity

a) Define Ambiguity in CFG with proper examples.

Ans: mid 9

b) Ambiguity – Operator Precedence

Ans: Ambiguous grammars (because of ambiguous operators) can be disambiguated according to the precedence and associativity rules.

$$E \rightarrow E+E \mid E * E \mid E^E \mid id \mid (E)$$

disambiguate the grammar
 precedence: $\wedge$ (right to left)
 $\downarrow$ * (left to right)
 + (left to right)

$$\begin{aligned} E &\rightarrow E+T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow G^F \mid G \\ G &\rightarrow id \mid (E) \end{aligned}$$

4. CFG

i. Definition of CFG(Context Free Grammar? Explain the different components of a CFG with an example.

Ans: mid 7

ii. Context Free Grammar Derivations Step and Example.

Ans: In a context-free grammar, we have:

- ☐ A finite set of terminals

- ☐ A finite set of non-terminals
- ☐ A finite set of production rules of the form

$$A \rightarrow \alpha$$

where A is a non-terminal and α is a string of terminals and non-terminals .

- ☐ A start symbol

Example:

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid - E$
 $E \rightarrow (E)$
 $E \rightarrow id$

Derivations:

$$E \Rightarrow E + E$$

- E+E derives from E
- we can replace E by E+E

$$E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + id$$

iii. Define Left most derivation with example.

Ans: If we always choose the left-most non-terminal in each derivation step, this derivation is called as **left-most derivation**.

Left-Most Derivation

$$E \xRightarrow{lm} -E \xRightarrow{lm} -(E) \xRightarrow{lm} -(E+E) \xRightarrow{lm} -(id+E) \xRightarrow{lm} -(id+id)$$

iv. Define Right most Derivation with example.

Ans: If we always choose the right-most non-terminal in each derivation step, this derivation is called as **right-most derivation**.

Right-Most Derivation

$$E \xRightarrow{rm} -E \xRightarrow{rm} -(E) \xRightarrow{rm} -(E+E) \xRightarrow{rm} -(E+id) \xRightarrow{rm} -(id+id)$$

v. Derivation:

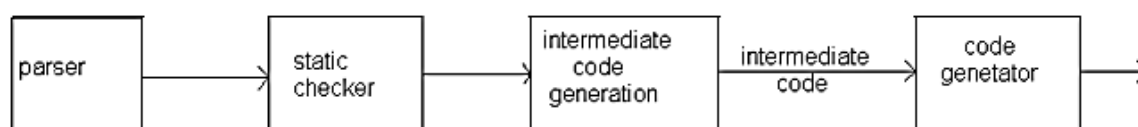
Using the CFG defined on the earlier slide, we can derive the string: 9 - 5 + 2 as follows:

$list \rightarrow list + digit$	P1 : $list \rightarrow list + digit$
$\rightarrow list - digit + digit$	P2 : $list \rightarrow list - digit$
$\rightarrow digit - digit + digit$	P3 : $list \rightarrow digit$
$\rightarrow 9 - digit + digit$	P4 : $digit \rightarrow 9$
$\rightarrow 9 - 5 + digit$	P4 : $digit \rightarrow 5$
$\rightarrow 9 - 5 + 2$	P4 : $digit \rightarrow 2$

5. Intermediate Code

i. Define Intermediate Code .

Ans: The source code of a program is translated into a form called intermediate code. **Intermediate Code (IC)** is a **machine-independent code** generated by the compiler between the **source code** and **machine code**.

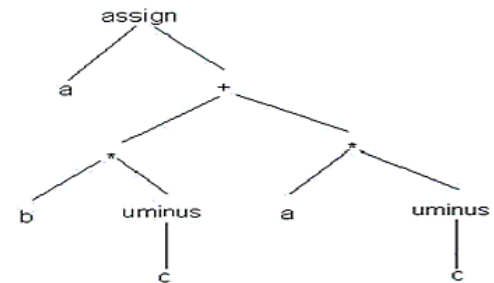


ii. Define Semantic Tree/ Syntax Tree.

Ans: A syntax tree depicts the natural hierarchical structure of a source program.

Ex. A syntax tree for the assignment statement

$a := b * -c + a * -c$



iii. Quadruples vs Triples vs Indirect Triples.

Ans:

Feature	Quadruples	Triples	Indirect Triples
Number of fields	4	3	3+ pointer table
Format	(op, arg1, arg2, result)	(op, arg1, arg2)	Pointer → Triple
Temporary variables	Required	Not required	Not required
Reference to previous result	Temporary variable	Statement number	Pointer/statement number
Code movement	Easy	Difficult	Easy
Memory requirement	More	Less	More than Triples
Implementation	Simple	Simple	More Complex

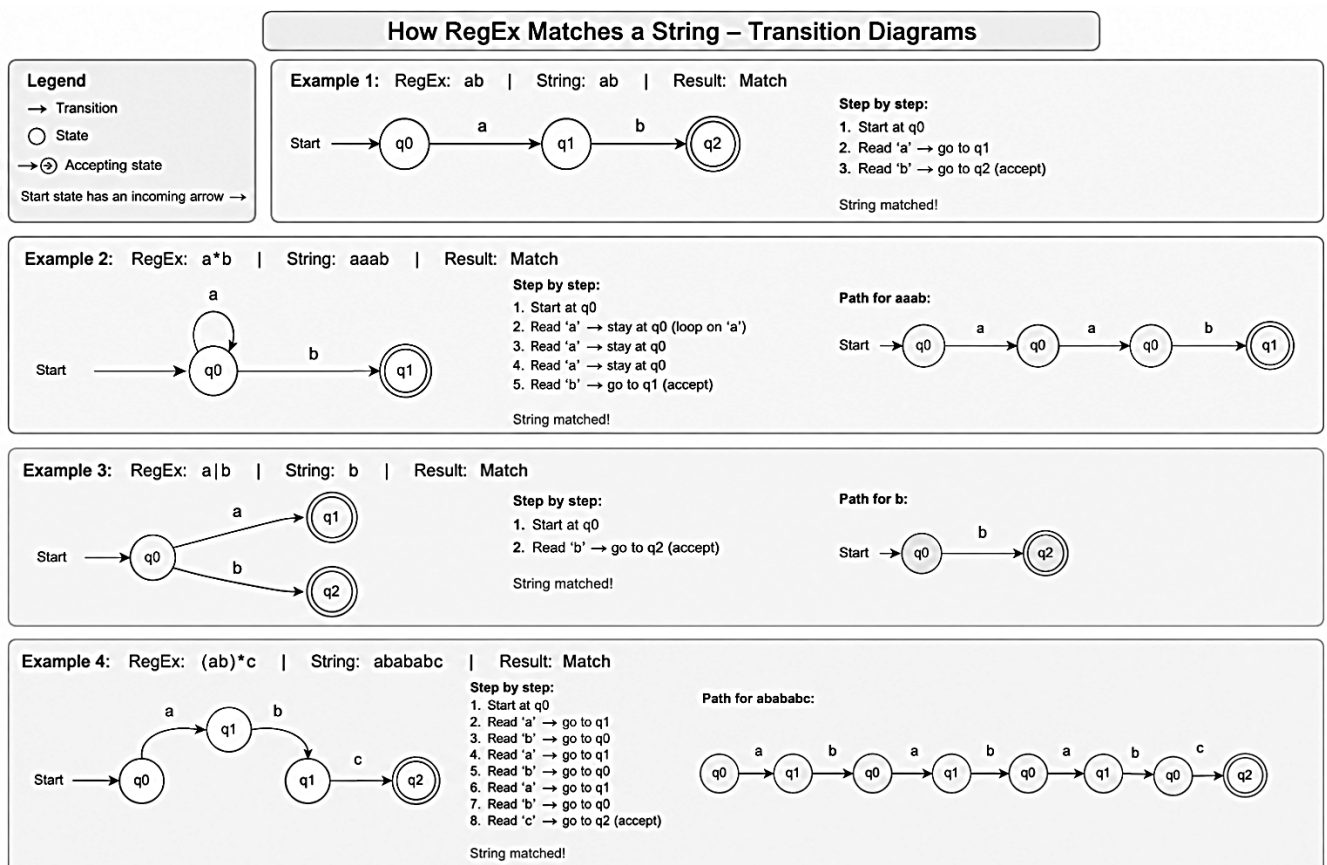
Mathematical

1. Generate regular expressions for matching certain string formats.

Ans: mid Technical question 02.

2. Draw transition diagrams for how Regular Expression matches a string.

Ans:



A transition diagram contains:

- **Circle** → a state
- **Arrow** → transition from one state to another
- **Arrow labeled a, b, etc.** → the input character that causes the transition
- **Double circle** → **accepting/final state**
- **Start arrow** → starting state

The machine reads the input string **one character at a time** and follows the corresponding arrows.

3. Check a grammar for ambiguity

Ans: mid technical question 3

4. Eliminate left recursion and left factoring from a given grammar

5. Remove all types of ambiguity from a grammar (remove op ambiguity, remove left recursion, implement left factoring)

Implement Left Factoring:

- A predictive parser (a top-down parser without backtracking) insists that the grammar must be *left-factored*.

$$\text{stmt} \rightarrow \text{if expr then stmt else stmt} \mid$$
$$\text{if expr then stmt}$$

- when we see if, we cannot know which production rule to choose to re-write *stmt* in the derivation
- In general, $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$
- when processing α we cannot know whether expand A to $\alpha\beta_1$ or A to $\alpha\beta_2$
- But, if we re-write the grammar as follows

$$A \rightarrow \alpha A'$$
$$A' \rightarrow \beta_1 \mid \beta_2$$

so, we can immediately expand A to $\alpha A'$

6. Generate Three Address Code from statements and represent using quadruples/ triples/ indirect triples.